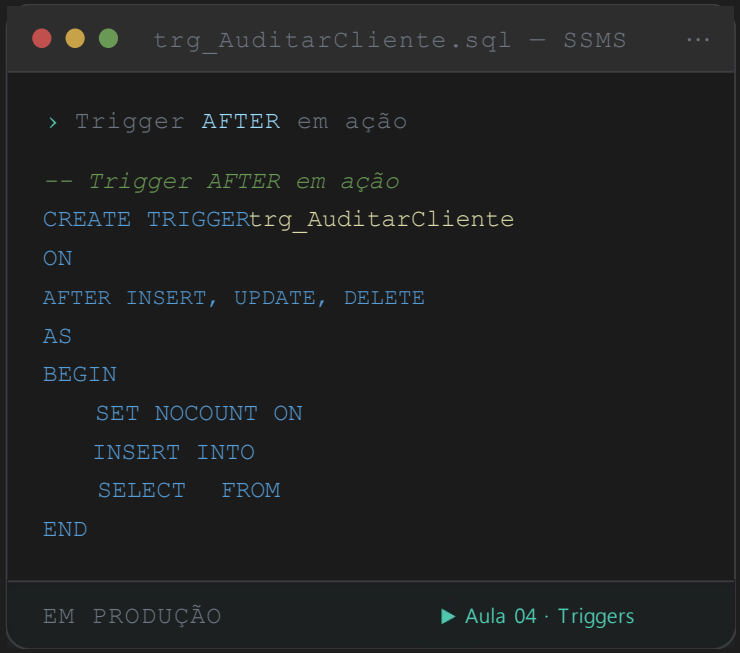
BANCO DE DADOS · AULA 04

MSSQL · T-SQL

Triggers no SQL Server

Regras de Negócio, Auditoria e Controle de Integridade · Boas Práticas e Práticas e Impactos no Desempenho.

⚠ OBJETO PODEROSO & PERIGOSO



SQL SERVER · TRIGGERS · AULA 04

SUMÁRIO

Agenda da Aula

Dois blocos cobrindo implementação prática e impacto operacional das triggers.

4.1

Implementação de Triggers para Regras de Negócio, Auditoria e Auditoria e Integridade

- ›O que são triggers e tipos (DML, DDL, LOGON)
- ›Anatomia do CREATE TRIGGER
- ›Tabelas virtuais inserted e deleted
- ›AFTER vs INSTEAD OF
- ›Triggers para regras de negócio
- ›Triggers para auditoria completa
- ›Controle de integridade referencial
- ›DDL Triggers para esquema
- ›Exemplo integrador

4.2

Boas Práticas e Impactos no Desempenho

- ›Por que triggers são problemáticas
- ›Custos de CPU, memória e locks
- ›Cascata de triggers
- ›Alternativas (constraints, computed columns, Change Data Capture)
- ›Padrões de otimização
- ›Monitoramento
- ›Checklist profissional

SEÇÃO

4.1

CHAPTER

Implementação de Triggers para Regras de Negócio, Auditoria e Integridade

Do conceito aos padrões profissionais — quando e como usar triggers com segurança.

01 → O que são triggers

02 → Tipos (DML/DDL/LOGON)

03 → Tabelas inserted/deleted

04 → AFTER vs INSTEAD OF

05 → Regras de negócio

06 → Auditoria

07 → Integridade referencial

08 → DDL Triggers

09 → Exemplo integrador

CONCEITO FUNDAMENTAL

O que são Triggers?

Triggers são stored procedures especiais executadas automaticamente pelo SQL Server em resposta a eventos específicos. Diferentemente das SPs comuns, não há EXEC explícito — a trigger dispara como efeito colateral de operações DML (INSERT/UPDATE/DELETE), DDL (CREATE/ALTER/DROP) ou LOGON. Vivem no contexto da transação do usuário que as disparou.

São o único mecanismo que garante regras de negócio independentemente da origem do dado (aplicação, (aplicação, script ad-hoc, job batch, integração). Mas essa mesma invisibilidade é o que as torna difíceis de difíceis de debugar e perigosas em ambientes não controlados.

CARACTERÍSTICAS

- ›Disparo automático (sem EXEC)
- ›Associadas a tabela/view (DML) ou database/server (DDL)
- ›Executadas no contexto da transação do caller
- ›Podem acessar tabelas virtuais inserted/deleted
- ›Suportam múltiplas por evento (ordem via sp_settriggerorder)
- ›Não suportam parâmetros explícitos

TIPOS DE TRIGGER

DML · DATA MANIPULATION

Disparam em `INSERT`, `UPDATE`, `DELETE` em tabela ou view.

Subtipos: `AFTER` · `INSTEAD OF` · `FOR` (sinônimo de AFTER)
Quando usar: Auditoria, validações complexas, derivação de dados

DDL · DATA DEFINITION

Disparam em `CREATE`, `ALTER`, `DROP` em nível de database ou servidor.

Exemplos: `CREATE_TABLE` · `ALTER_PROCEDURE` · `DROP_VIEW`
Quando usar: Auditoria de schema, governança, prevenir mudanças críticas

LOGON · AUTHENTICATION

Disparam quando uma `sessão` é estabelecida. Recebem `EVENTDATA()` com dados de conexão.

Quando usar: Auditoria de acesso, restrição por IP/horário, limite de conexões concorrentes
Requisitos: Apenas SQL Server 2005+ · `VIEW SERVER STATE`

SINTAXE

Anatomia do CREATE TRIGGER

Uma trigger DML é composta por cabeçalho (CREATE TRIGGER + nome + tabela + evento), corpo (AS BEGIN … END) e finalizador (GO). Cada parte tem regras importantes.

COMPONENTES DO CABEÇALHO

- 01 CREATE TRIGGER — criação (ou CREATE OR ALTER SQL 2016+)
- 02 Nome com schema: dbo.trg_Nome
- 03 ON dbo.Tabela — tabela ou view alvo
- 04 AFTER | INSTEAD OF — quando dispara
- 05 INSERT, UPDATE, DELETE — eventos (um ou mais)
- 06 AS — início do corpo
- 07 WITH ENCRYPTION / SCHEMABINDING — opções

BOAS PRÁTICAS NO CORPO

- › SET NOCOUNT ON — primeira linha
- › Acessar inserted/deleted via SELECT INTO @tabela
- › Verificar se houve mudança real: IF UPDATE(coluna)
- › Tratar erros via TRY/CATCH (mas ROLLBACK em CATCH)
- › Documentar via sp_addextendedproperty

✦trg_AuditarClientes.sql · T-SQL

```sql
-- Anatomia completa
USE
GO

CREATE OR ALTER TRIGGER  trg_AuditarClientes
ON
AFTER INSERT, UPDATE, DELETE
-- WITH ENCRYPTION, SCHEMABINDING  -- opcional
AS
BEGIN
   SET NOCOUNT ON  -- sempre!

   -- Detectar tipo de operação
   DECLARE @Operacao CHAR(1)
   IF EXISTS (SELECT 1 FROM         AND EXISTS (SELECT 1 FROM
       SET @Operacao  'U'  -- UPDATE
   ELSE IF EXISTS (SELECT 1 FROM
       SET @Operacao  'I'  -- INSERT
   ELSE IF EXISTS (SELECT 1 FROM
       SET @Operacao  'D'  -- DELETE

   -- Verificar se coluna específica mudou (apenas em UPDATE)
   IF @Operacao  'U' AND UPDATE
   BEGIN
      INSERT INTO


       SELECT
                    'U'

          SUSER_SNAME   GETDATE
       FROM
       INNER JOIN         ON
       WHERE                    -- só se mudou
   END
END
GO

-- Documentação
EXEC sp_addextendedproperty
   @name N'Descricao'
   @value N'Audita mudanças de Status em Clientes'
   @level0type    N'SCHEMA' @level0name   N'dbo'
   @level1type    N'TABLE' @level1name   N'Clientes'
   @level2type    N'TRIGGER' @level2name   N'trg_AuditarClientes'
GO
```

CONCEITO CRÍTICO

Tabelas Virtuais `inserted` e `deleted`

Quando uma trigger DML dispara, o SQL Server cria duas tabelas virtuais em memória, disponíveis disponíveis apenas dentro da trigger. São a ÚNICA forma de acessar os dados antes e depois da depois da operação que disparou o evento.

CARACTERÍSTICAS DAS TABELAS

- ›Schema idêntico à tabela base
- ›Existem apenas dentro da trigger
- ›Em memória (não persistem)
- ›Para UPDATE: inserted = novos, deleted = antigos
- ›Para INSERT: só inserted tem dados
- ›Para DELETE: só deleted tem dados
- ›Múltiplas linhas: SEMPRE iterar via set-based

PADRÕES DE ACESSO

- →SELECT * FROM inserted — novos valores
- →SELECT * FROM deleted — antigos valores
- →JOIN inserted i ON d.ClienteId = i.ClienteId
- →IF EXISTS (SELECT 1 FROM inserted WHERE ...)

⚠ ARMADILHA CLÁSSICA
Iniciantes assumem 1 linha — ERRADO!
INSERT pode trazer 10.000 linhas (ex: `INSERT INTO ... SELECT`)
SEMPRE tratar como conjunto

✦trg_DemoTabelas.sql · T-SQL

```sql
-- Demo: como inserted/deleted se comportam

CREATE OR ALTER TRIGGER trg_DemoTabelas
ON
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
   SET NOCOUNT ON

    -- Conta linhas afetadas
    DECLARE @Inserted INT  @@ROWCOUNT
    DECLARE @Inserts INT  SELECT COUNT  FROM
    DECLARE @Deletes INT  SELECT COUNT  FROM

    PRINT 'Linhas em inserted: CAST @Inserts AS VARCHAR
    PRINT 'Linhas em deleted:  CAST @Deletes AS VARCHAR

    -- Caso 1: INSERT (só inserted tem dados)
    IF @Inserts 0 AND @Deletes 0
    BEGIN
       INSERT INTO

       SELECT       'I'      SUSER_SNAME  GETDATE
        FROM        -- set-based, suporta N linhas
    END

    -- Caso 2: DELETE (só deleted tem dados)
    IF @Deletes 0 AND @Inserts 0
    BEGIN
       INSERT INTO

       SELECT       'D'      SUSER_SNAME  GETDATE
        FROM
    END

    -- Caso 3: UPDATE (ambas têm dados)
    IF @Inserts 0 AND @Deletes 0
    BEGIN
       INSERT INTO

       SELECT        'U'

            SUSER_SNAME  GETDATE
        FROM
       INNER JOIN       ON
        WHERE                -- só se mudou
    END
END
GO
```

AFTER vs INSTEAD OF — Quando Usar Cada

A escolha entre AFTER e INSTEAD OF muda fundamentalmente o comportamento da trigger.

AFTER · PADRÃO

Dispara APÓS a operação DML ser processada e gravada (mas antes do commit). Os dados já dados já estão na tabela.

QUANDO USAR

- Auditoria (95% dos casos)
- Validação pós-operacional
- Derivação de dados (calcular totais, etc.)
- Notificações assíncronas (Service Broker)

VANTAGENS

- Simples de raciocinar (já aconteceu)
- Pode ler a tabela atualizada
- ROLLBACK desfaz a operação

LIMITAÇÕES

- Só em tabelas (não em views com CHECK OPTION)
- Não pode substituir a operação

```
CREATE TRIGGER trg_AfterInsert
ON dbo.Clientes
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON
    INSERT INTO dbo.LogClientes (ClienteId, Data)
    SELECT ClienteId, GETDATE()
    FROM inserted
END
```

INSTEAD OF · SUBSTITUI

Dispara NO LUGAR da operação DML. A operação original NÃO é executada — a trigger decide o trigger decide o que fazer.

QUANDO USAR

- Views complexas (atualizar múltiplas tabelas base)
- Simular operações não suportadas (ex: DELETE em view com JOIN)
- Validar e reescrever dados antes de gravar
- Cross-table updates a partir de view

VANTAGENS

- Funciona em views (mesmo com JOINs)
- Permite reescrever lógica de DML
- Mais flexível que AFTER

LIMITAÇÕES

- Mais complexo de raciocinar
- Pode causar comportamento inesperado
- Não suportado em algumas constraints

```
CREATE TRIGGER trg_InsteadInsert
ON dbo.vw_ClienteCompleto
INSTEAD OF INSERT
AS
BEGIN
    SET NOCOUNT ON
    -- Separa dados em duas tabelas base
    INSERT INTO dbo.Clientes (Nome, Email)
    SELECT Nome, Email FROM inserted

    INSERT INTO dbo.Enderecos (ClienteId, Cidade)
    SELECT SCOPE_IDENTITY(), Cidade
    FROM inserted
END
```

Regra prática: comece com AFTER. Use INSTEAD OF apenas em views ou quando precisar reescrever a operação.

CASO DE USO 1/3

Triggers para Regras de Negócio

Quando a regra de negócio é complexa demais para CHECK CONSTRAINT e precisa CONSTRAINT e precisa de lógica procedural (múltiplas tabelas, cálculos, integrações), triggers são a única forma de garanti-la independentemente da independentemente da origem do dado.

EXEMPLOS TÍPICOS

- Saldo de conta não pode ficar negativo
- Cliente em status 'BLOQUEADO' não pode comprar
- Pedido só pode ser cancelado antes do envio
- Desconto máximo por faixa de cliente
- Estoque mínimo deve ser respeitado

VANTAGENS

- Garantia independente da origem (app, script, integração)
- Lógica centralizada no banco
- Não pode ser bypassada

ATENÇÃO

- Prefira CHECK constraint quando possível (mais simples)
- Documentar regra de negócio via comment
- Mensagens de erro claras via THROW
- Considerar performance (trigger roda a cada DML)

✦triggers_regras_negocio.sql · T-SQL

```sql
-- Trigger: impedir venda para cliente bloqueado
CREATE OR ALTER TRIGGER trg_BloquearVendaCliente
ON dbo.Vendas
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS (
        SELECT 1
        FROM inserted i
        INNER JOIN dbo.Clientes c
            ON c.ClienteId = i.ClienteId
        WHERE c.Status = 'BLOQUEADO'
    )
    BEGIN
        -- ROLLBACK desfaz o INSERT/UPDATE original
        ROLLBACK TRANSACTION;

        -- Erro amigável para o usuário
        THROW 50001,
            'Não é possível registrar venda para cliente BLOQUEADO. '
            'Contate o departamento de cobrança.',
            1;
    END
END;
GO

-- Trigger: validar saldo de conta
CREATE OR ALTER TRIGGER trg_ValidarSaldo
ON dbo.Movimentacoes
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS (
        SELECT 1
        FROM inserted i
        INNER JOIN dbo.Contas c ON c.ContaId = i.ContaId
        WHERE c.Saldo + i.Valor < 0 -- ficaria negativo
    )
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50002,
            'Saldo insuficiente para esta movimentação.',
            1;
    END
END;
GO

-- Trigger: desconto máximo por faixa
CREATE OR ALTER TRIGGER trg_ValidarDesconto
ON dbo.ItensPedido
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    IF UPDATE(Desconto)
    AND EXISTS (
        SELECT 1
        FROM inserted i
        INNER JOIN dbo.Clientes c ON c.ClienteId = i.ClienteId
        INNER JOIN dbo.FaixasDesconto fd
            ON fd.Faixa = c.Faixa
        WHERE i.Desconto > fd.DescontoMaximo
    )
    BEGIN
        ROLLBACK;
        THROW 50003,
            'Desconto excede o máximo permitido para a faixa do cliente.',
            1;
    END
END;
GO
```

CASO DE USO 2/3

Triggers para Auditoria

Auditoria é o uso mais comum e justificável de triggers. Quando se precisa rastrear precisa rastrear quem alterou o quê, quando, e qual era o valor anterior — para — para compliance (LGPD, SOX, PCI-DSS) ou simplesmente para investigar investigar incidentes.

ESTRATÉGIAS DE AUDITORIA

- Tabela de log dedicada — schema espelho + colunas de auditoria
- Colunas na mesma tabela — ModifiedBy, ModifiedAt (limitada)
- Temporal tables (SQL 2016+) — alternativa moderna sem trigger
- Change Data Capture (CDC) — assíncrono, via log
- Change Tracking (CT) — leve, só IDs

COLUNAS TÍPICAS DE LOG

- Id (auto-numerado)
- PK original (ClienteId, etc.)
- Operacao (I/U/D)
- Coluna alterada (opcional, granular)
- ValorAntigo, ValorNovo
- Usuario (SUSER_SNAME)
- DataAlteracao (GETDATE)
- Aplicacao (APP_NAME)
- IP (via CONNECTION property)

✦triggers_auditoria.sql · T-SQL

```sql
-- 1. Criar tabela de auditoria (schema espelho)
CREATE TABLE dbo.Auditoria_Clientes
    AuditId   INT IDENTITY PRIMARY KEY
    Operacao CHAR(1) NOT NULL-- I/U/D
    ClienteId INT NOT NULL
    Nome_Antigo VARCHAR(100)
    Nome_Novo VARCHAR(100)
    Email_Antigo VARCHAR(150)
    Email_Novo VARCHAR(150)
    Usuario SYSNAME DEFAULT NULL
    Aplicacao VARCHAR(128)
    DataEvento DATETIME2 NOT NULL DEFAULT GETDATE

GO

-- 2. Trigger genérica de auditoria
CREATE OR ALTER TRIGGER trg_AuditarClientes
ON dbo.Clientes
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
 SET NOCOUNT ON
  DECLARE @Op CHAR(1)
  DECLARE @User SYSNAME = SUSER_SNAME
  DECLARE @App VARCHAR(128) = APP_NAME

   -- Detecta operação
  IF EXISTS (SELECT 1 FROM inserted) AND EXISTS (SELECT 1 FROM deleted)
     SET @Op = 'U'
 ELSE IF EXISTS (SELECT 1 FROM inserted)
     SET @Op = 'I'
  ELSE
     SET @Op = 'D'

   -- Loga INSERT ou UPDATE
  IF @Op IN ('I','U')
   INSERT INTO dbo.Auditoria_Clientes
       (Operacao, ClienteId, Nome_Antigo, Nome_Novo
       ,Email_Antigo, Email_Novo, Usuario, Aplicacao)
    SELECT @Op
          ,i.ClienteId
          ,d.Nome, i.Nome
          ,d.Email, i.Email
          ,@User, @App
     FROM inserted i
     LEFT JOIN deleted d ON d.ClienteId = i.ClienteId

  -- Loga DELETE
  IF @Op = 'D'
   INSERT INTO dbo.Auditoria_Clientes
       (Operacao, ClienteId, Nome_Antigo, Nome_Novo
       ,Email_Antigo, Email_Novo, Usuario, Aplicacao)
    SELECT 'D'
          ,d.ClienteId
          ,d.Nome, NULL
          ,d.Email, NULL
          ,@User, @App
     FROM deleted d
END
GO

-- 3. Alternativa moderna: Temporal Tables (SQL Server 2016+)
CREATE TABLE dbo.Clientes
   ClienteId INT PRIMARY KEY
   Nome VARCHAR(100)
   Email VARCHAR(150)
  SysStartTime DATETIME2 GENERATED ALWAYS AS ROW START NOT NULL
  SysEndTime DATETIME2 GENERATED ALWAYS AS ROW END NOT NULL
   PERIOD FOR SYSTEM_TIME (SysStartTime, SysEndTime)

WITH (SYSTEM_VERSIONING = ON (HISTORY_TABLE = dbo.ClientesHistory))
-- SEM trigger necessária! SQL Server mantém histórico automaticamente
GO
```

CASO DE USO 3/3

Triggers para Controle de Integridade

Quando a integridade referencial vai além do que FOREIGN KEY suporta — updates — updates em cascata customizados, validações cross-table complexas, ou ou derivação de dados dependentes — triggers são a ferramenta.

FOREIGN KEY	simples, declarativa, performática. Use sempre que possível.
Trigger	para casos além de FK — cascata customizada, derivação, derivação, validação procedural.

CASOS QUE PRECISAM DE TRIGGER

- Atualizar tabela derivada (totais, agregações)
- Manter tabela desnormalizada em sincronia
- Cascata multi-tabela com regras
- Sincronizar com sistema externo (cuidado!)
- Quando FK com ON DELETE CASCADE não é suficiente

BOAS PRÁTICAS

- Sempre set-based
- Verificar IF UPDATE(coluna) antes de derivar
- Considerar computed columns como alternativa
- Para totais: preferir view indexada ou recompute sob demanda

✦triggers_integridade.sql · T-SQL

```sql
-- Trigger: manter total do pedido ao inserir item
CREATE OR ALTER TRIGGER dbo.trg_AtualizarTotalPedido
ON dbo.ItensPedido
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
  SET NOCOUNT ON

  -- Coleta IDs de pedidos afetados (inserted + deleted)
  DECLARE @PedidosAfetados TABLE (PedidoId INT PRIMARY KEY)

  INSERT INTO @PedidosAfetados (PedidoId)
  SELECT PedidoId FROM inserted
  UNION
  SELECT PedidoId FROM deleted

  -- Recalcula total para cada pedido afetado (set-based!)
  UPDATE p
  SET TotalPedido = (
      SELECT SUM(ip.Quantidade * ip.Preco)
      FROM dbo.ItensPedido ip
      WHERE ip.PedidoId = p.PedidoId
  ),
      DataModificacao = GETDATE()
  FROM dbo.Pedidos p
  INNER JOIN @PedidosAfetados pa ON pa.PedidoId = p.PedidoId
END
GO

-- Trigger: cascata customizada — ao excluir cliente,
-- arquivar pedidos em vez de deletar
CREATE OR ALTER TRIGGER dbo.trg_ArquivarAoExcluir
ON dbo.Clientes
INSTEAD OF DELETE
AS
BEGIN
  SET NOCOUNT ON

  -- 1. Marca pedidos como arquivados
  UPDATE dbo.Pedidos
  SET Status = 'ARQUIVADO',
      DataArquivamento = GETDATE()
  WHERE ClienteId IN (SELECT ClienteId FROM deleted)

  -- 2. Move cliente para tabela de arquivamento
  INSERT INTO dbo.ClientesArquivados
      (ClienteId, Nome, Email, DataArquivamento, Motivo)
  SELECT ClienteId, Nome, Email, GETDATE(), 'Excluido por trigger'
  FROM deleted

  -- 3. Finalmente, deleta o cliente original
  DELETE FROM dbo.Clientes
  WHERE ClienteId IN (SELECT ClienteId FROM deleted)
END
GO

-- Alternativa: FOREIGN KEY com CASCADE (preferir quando simples)
/*
ALTER TABLE dbo.ItensPedido
  ADD CONSTRAINT FK_Itens_Pedido
FOREIGN KEY (PedidoId) REFERENCES dbo.Pedidos(PedidoId)
  ON DELETE CASCADE ON UPDATE CASCADE
*/
```

TRIGGER DDL

DDL Triggers — Governança de Schema

DDL triggers disparam em resposta a CREATE/ALTER/DROP em nível de database ou servidor. ou servidor. Não acessam dados — recebem EVENTDATA() com XML descrevendo a operação. descrevendo a operação. Essencial para auditoria e governança em ambientes regulados. regulados.

ESCOPO

- ON DATABASE — eventos de banco (CREATE_TABLE, ALTER_PROC, etc.)
- ON ALL SERVER — eventos de servidor (CREATE_LOGIN, ALTER_DATABASE)

EVENTDATA() RETORNA XML COM

- EventType (CREATE_TABLE, ALTER_PROCEDURE, etc.)
- PostTime
- SPID
- ServerName
- LoginName
- SqlCommand (T-SQL executado)

CASOS DE USO

- Auditoria de mudanças de schema (compliance)
- Bloquear DROP em tabelas críticas
- Forçar naming convention
- Versionamento automático (gerar script)

ATENÇÃO

- Não dispara em TRUNCATE (use TRUNCATE_TABLE event)
- Pode causar bloqueio de operations críticas
- Logar em tabela dedicada, não em arquivo

✦ddl_triggers.sql · T-SQL

```sql
-- 1. Auditoria completa de DDL
CREATE TABLE dbo.LogDDL
(
    LogId       INT IDENTITY PRIMARY KEY
    EventType   VARCHAR(100)
    ObjectName  VARCHAR(256)
    ObjectType  VARCHAR(32)
    SqlCommand  VARCHAR(MAX)
    LoginName   SYSNAME
    UsuarioHost SYSNAME
    DataEvento  DATETIME2
)
GO

CREATE OR ALTER TRIGGER trg_AuditarDDL
ON DATABASE
FOR DDL_DATABASE_LEVEL_EVENTS -- todos eventos DDL
AS
BEGIN
    SET NOCOUNT ON

    DECLARE @EventData XML = EVENTDATA()

    INSERT INTO dbo.LogDDL
        (EventType, ObjectName, ObjectType
         SqlCommand, LoginName, UsuarioHost, DataEvento)
    VALUES
        (@EventData.value('(/EVENT_INSTANCE/EventType)[1]', 'VARCHAR(100)'),
         @EventData.value('(/EVENT_INSTANCE/ObjectName)[1]', 'VARCHAR(256)'),
         @EventData.value('(/EVENT_INSTANCE/ObjectType)[1]', 'VARCHAR(32)'),
         @EventData.value('(/EVENT_INSTANCE/TSQLCommand/CommandText)[1]', 'NVARCHAR(MAX)'),
         @EventData.value('(/EVENT_INSTANCE/LoginName)[1]', 'SYSNAME'),
         HOST_NAME(),
         GETDATE())
END
GO

-- 2. Bloquear DROP TABLE em produção
CREATE OR ALTER TRIGGER trg_BloquearDropTable
ON DATABASE
FOR DROP_TABLE
AS
BEGIN
    SET NOCOUNT ON
    DECLARE @Obj VARCHAR(256) =
        EVENTDATA().value('(/EVENT_INSTANCE/ObjectName)[1]', 'VARCHAR(256)')

    -- Permite apenas em horário de manutenção (janela 22h-23h)
    IF DATEPART(HOUR, GETDATE()) NOT BETWEEN 22 AND 23
    BEGIN
        ROLLBACK
        THROW 50100,
            'DROP TABLE só permitido em janela de manutenção (22h-23h).',
            1
    END
END
GO

-- 3. Forçar naming convention em procedures
CREATE OR ALTER TRIGGER trg_ValidarNamingProc
ON DATABASE
FOR CREATE_PROCEDURE, ALTER_PROCEDURE
AS
BEGIN
    SET NOCOUNT ON
    DECLARE @ProcName VARCHAR(256) =
        EVENTDATA().value('(/EVENT_INSTANCE/ObjectName)[1]', 'VARCHAR(256)')

    IF LEFT(@ProcName, 4) <> 'usp_'
    BEGIN
        ROLLBACK
        THROW 50101,
            'Procedures devem seguir padrão usp_Nome.',
            1
    END
END
GO
```

TRIGGER LOGON

LOGON Triggers — Controle de Acesso

LOGON triggers disparam quando uma sessão é estabelecida com o SQL Server. São o mecanismo mais mecanismo mais cedo possível para auditar ou restringir conexões — antes mesmo do login completar. login completar.

CARACTERÍSTICAS

- Escopo: `ON ALL SERVER` (não há LOGON por database)
- Recebem `EVENTDATA()` com IP, login, application name
- ROLLBACK cancela a conexão (login falha)
- Disparam APÓS autenticação, ANTES de receber qualquer comando
- Requer permissão CONTROL SERVER ou ALTER ANY CONNECTION

CASOS DE USO TÍPICOS

- Bloquear conexões fora de horário comercial
- Limitar conexões por login (limite concorrente)
- Restringir por IP (whitelist/blacklist)
- Auditar todas as conexões
- Direcionar para database específico

> ⚠ PERIGO
>
> - Trigger com bug = ninguém consegue logar
> - Sempre testar em dev antes de produção
> - Manter conexão admin aberta como fallback

✦ logon_triggers.sql · T-SQL

```sql
-- 1. Auditoria de logons
CREATE TABLE dboAuditoriaLogon
    LogonId   INT IDENTITY PRIMARY KEY
    LoginNameSYSNAME
   ClientHostVARCHAR48
   ApplicationVARCHAR128
    DataLogonDATETIME2
    Sucesso  BIT -- 0 se bloqueado por trigger
GO

-- Trigger: bloquear fora de horário comercial
CREATE OR ALTER TRIGGER trg_BloquearForaHorario
ON ALL SERVER
FOR LOGON
AS
BEGIN
    DECLARE@EventData XMLEVENTDATA
    DECLARE@LoginName SYSNAME
      @EventDatavalue(/EVENT_INSTANCE/PostTime)[1]
    DECLARE@Hora INTDATEPART HOURGETDATE
    DECLARE@DiaSemana INTDATEPART WEEKDAYGETDATE
    DECLARE@Login NVARCHAR128
      @EventDatavalue(/EVENT_INSTANCE/LoginName)[1]

    -- Exceção: sa e admin
    IF@Login IN 'sa''admin_sql''NT SERVICE\SQLSERVERAGENT'
      RETURN

    -- Bloqueia fora de 7h-19h em dias úteis
    IF@DiaSemana BETWEEN 2 AND 6 -- seg-sex
    AND @Hora 7 OR@Hora 19
    BEGIN
      ROLLBACK -- cancela a conexão!

      -- Loga tentativa
      INSERT INTOmasterdboAuditoriaLogon
          LoginNameClientHostApplicationDataLogonSucesso
      VALUES
          @Login
          @EventDatavalue(/EVENT_INSTANCE/ClientHost)[1]
          @EventDatavalue(/EVENT_INSTANCE/ApplicationName)[1]
          GETDATE
          0
    END
END
GO

-- 2. Limitar conexões concorrentes por login
CREATE OR ALTER TRIGGER trg_LimiteConexoes
ON ALL SERVER
FOR LOGON
AS
BEGIN
    DECLARE@Login NVARCHAR128
      EVENTDATAvalue(/EVENT_INSTANCE/LoginName)[1]

    -- Usuário 'app_user' limitado a 5 conexões simultâneas
    IF@Login'app_user'
    AND
      SELECT COUNT
        FROMsys.dm_exec_sessions
        WHERElogin_name@Login
          ANDis_user_process
          1
    5
    BEGIN
      ROLLBACK
      -- Logar razão...
    END
END
GO

-- Para desativar em caso de emergência:
-- DISABLE TRIGGER trg_BloquearForaHorario ON ALL SERVER;
```

EXEMPLO INTEGRADOR

Auditoria Profissional de Clientes

Trigger completa para auditoria **multi-coluna**, com detecção detecção de operação, captura de contexto (usuário, app, IP), app, IP), e mensagem de erro amigável.

RECURSOS COMBINADOS

- ›AFTER INSERT/UPDATE/DELETE
- ›Detecção automática de operação
- ›Log granular por coluna alterada
- ›Contexto completo (user/app/host)
- ›Set-based em todo o corpo
- ›Tratamento de erros
- ›Documentação via extended property

PADRÃO REUTILIZÁVEL

Uma única trigger por tabela — adaptar lista de colunas colunas auditadas.

trg_AuditoriaClientes.sql

```sql
-- 1. Tabela de auditoria genérica (uma linha por coluna alterada)
CREATE TABLE dbo.Auditoria (
AuditId       INT IDENTITY PRIMARY KEY,
Tabela        SYSNAME NOT NULL,
Operacao      CHAR(1) NOT NULL,
RegistroId    NVARCHAR(50) NOT NULL,
Coluna        SYSNAME NOT NULL,
ValorAntigo   NVARCHAR(MAX),
ValorNovo     NVARCHAR(MAX),
Usuario       SYSNAME NOT NULL,
Aplicacao     NVARCHAR(128),
Host          NVARCHAR(128),
DataEvento    DATETIME2 NOT NULL DEFAULT(GETDATE())
);
GO
-- 2. Trigger genérica para auditoria multi-coluna
CREATE OR ALTER TRIGGER dbo.trg_AuditoriaClientes
ON dbo.Clientes
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
SET NOCOUNT ON;
SET XACT_ABORT ON;
BEGIN TRY
DECLARE @Op CHAR(1);
IF EXISTS (SELECT 1 FROM inserted) AND EXISTS (SELECT 1 FROM deleted)
SET @Op = 'U';
ELSE IF EXISTS (SELECT 1 FROM inserted)
SET @Op = 'I';
ELSE
SET @Op = 'D';
IF @Op = 'I'
INSERT INTO dbo.Auditoria (Tabela, Operacao, RegistroId, Coluna,
ValorAntigo, ValorNovo, Usuario, Aplicacao, Host)
SELECT 'Clientes', 'I',
CAST(i.ClienteId AS NVARCHAR(50)), 'Nome',
NULL, i.Nome,
SUSER_SNAME(), APP_NAME(), HOST_NAME()
FROM inserted i;
IF @Op = 'D'
INSERT INTO dbo.Auditoria (Tabela, Operacao, RegistroId, Coluna,
ValorAntigo, ValorNovo, Usuario, Aplicacao, Host)
SELECT 'Clientes', 'D',
CAST(d.ClienteId AS NVARCHAR(50)), 'Nome',
d.Nome, NULL,
SUSER_SNAME(), APP_NAME(), HOST_NAME()
FROM deleted d;
IF @Op = 'U'
BEGIN
-- Audita apenas colunas que mudaram
IF UPDATE(Nome)
INSERT INTO dbo.Auditoria (Tabela, Operacao, RegistroId, Coluna,
ValorAntigo, ValorNovo, Usuario, Aplicacao, Host)
SELECT 'Clientes', 'U', CAST(i.ClienteId AS NVARCHAR(50)), 'Nome',
d.Nome, i.Nome,
SUSER_SNAME(), APP_NAME(), HOST_NAME()
FROM inserted i
INNER JOIN deleted d ON d.ClienteId = i.ClienteId
WHERE ISNULL(i.Nome, '') <> ISNULL(d.Nome, '');
IF UPDATE(Email)
INSERT INTO dbo.Auditoria (Tabela, Operacao, RegistroId, Coluna,
ValorAntigo, ValorNovo, Usuario, Aplicacao, Host)
SELECT 'Clientes', 'U', CAST(i.ClienteId AS NVARCHAR(50)), 'Email',
d.Email, i.Email,
SUSER_SNAME(), APP_NAME(), HOST_NAME()
FROM inserted i
INNER JOIN deleted d ON d.ClienteId = i.ClienteId
WHERE ISNULL(i.Email, '') <> ISNULL(d.Email, '');
END
END TRY
BEGIN CATCH
```

SEÇÃO

4.2

⚠ CRITICAL CHAPTER

Boas Práticas e Impactos das Triggers no Desempenho

O que toda equipe sênior sabe: triggers são custosas, invisíveis e podem destruir performance — performance — saiba quando usar e quando evitar.

01 → Por que triggers são problemáticas

02 → Custos de CPU / memória / locks

03 → Cascata de triggers

04 → Alternativas modernas

05 → Padrões de otimização

06 → Monitoramento

07 → Checklist profissional

RISCOS CONCRETOS

Por que Triggers são Problemáticas

Seis riscos concretos que toda equipe sênior conhece — e como reconhecê-los.

6 RISCOS · 1 REGRA
DIAGNÓSTICO RÁPIDO

01 · INVISIBILIDADE
Invisibilidade no Código

Triggers não aparecem no código da aplicação. Desenvolvedor INSERT e não sabe que dispara lógica. Bugs Bugs difíceis de rastrear. Sempre documentar via `sp_addextendedproperty` e comentar nos

02 · PERFORMANCE
Custo por Operação

Cada INSERT/UPDATE/DELETE paga: parse + compile da da trigger + execução. Em tabelas com 1000 inserts/segundo, `trigger pesada vira gargalo`. `SET NOCOUNT ON` ajuda mas não resolve tudo.

03 · CONCORRÊNCIA
Locks Estendidos

Trigger roda DENTRO da transação do caller. Locks são mantidos até o fim da trigger. Trigger lenta = locks longos = bloqueios em cascata = deadlocks.

04 · RECURSÃO
Cascata de Triggers

Trigger A em Tabela X faz UPDATE em Y → B em Y dispara → INSERT em Z → C dispara... Resultado: comportamento inesperado, deadlocks, e debugging impossível.

05 · DEBUGGING
Difícil Debug

PRINT dentro de trigger só aparece se executado diretamente. SSMS não debuga trigger passo-a-passo facilmente. Solução: logar em tabela de debug, ou usar `SQL Server Profiler` / Extended Events.

06 · DEPENDÊNCIA
Acoplamento Escondido

Trigger em Tabela X acessa Tabela Y. Renomear/dropar Y quebra a trigger. Constraint de FK alerta explicitamente; trigger não. Sempre gerar script completo ao alterar schema.

◆ Regra prática: se a regra de negócio puder ser implementada SEM trigger (constraint, app layer, computed column), prefira.

IMPACTO MENSURÁVEL

Custos de CPU, Memória e Locks

Cada trigger DML adiciona custo mensurável a cada operação. Em sistemas de alta carga, o impacto é direto: CPU%, memória do buffer pool, e duração de locks — todos aumentam.

CUSTOS TÍPICOS

- CPU: parse + compile da trigger (com cache de plano)
- Memória: planos em cache, tabelas inserted/deleted
- Locks: mantidos até o final da transação da trigger
- I/O: operações dentro da trigger geram I/O adicional
- Rede: SET NOCOUNT OFF gera tráfego extra

PADRÕES DE OTIMIZAÇÃO

- Sempre SET NOCOUNT ON
- Validar mudança real (IF UPDATE(col) + WHERE i.X <> d.X)
- Operações set-based (nunca cursor!)
- Evitar SELECT * em inserted/deleted
- Reduzir I/O: índices nas tabelas de log
- Operação assíncrona via Service Broker

MÉTRICAS A MONITORAR

- sys.dm_exec_trigger_stats (desde SQL 2008)
- sys.dm_exec_procedure_stats (inclui triggers)
- Wait statistics: LCK_M_* (lock waits)
- Query Store captura execução de triggers

otimizacao_e_diagnostico.sql T-SQL · DMVs

```sql
-- 1. Trigger OTIMIZADA: validar mudança antes de agir
CREATE OR ALTER TRIGGER dbo.trg_AuditarEmailCliente_Otima
ON dbo.Clientes
AFTER UPDATE
AS
BEGIN
SET NOCOUNT ON;   -- sempre!
-- Sai imediatamente se coluna Email não foi tocada
IF NOT UPDATE(Email)
RETURN;
-- Loga apenas se valor mudou de verdade
INSERT INTO dbo.LogEmails
(ClienteId, EmailAntigo, EmailNovo, DataAlteracao)
SELECT i.ClienteId, d.Email, i.Email, GETDATE()
FROM inserted i
INNER JOIN deleted d ON d.ClienteId = i.ClienteId
WHERE i.Email <> d.Email   -- mudança real
OR (i.Email IS NULL AND d.Email IS NOT NULL)
OR (i.Email IS NOT NULL AND d.Email IS NULL);
END
GO
-- 2. Top triggers por consumo de CPU
SELECT TOP (20)
DB_NAME(t.database_id) AS Banco,
OBJECT_SCHEMA_NAME(t.object_id, t.database_id) AS SchemaName,
OBJECT_NAME(t.object_id, t.database_id) AS TriggerName,
t.execution_count AS Execucoes,
t.total_worker_time / t.execution_count AS CpuMediaUs,
t.total_elapsed_time / t.execution_count AS DuracaoMediaUs,
t.total_logical_reads / t.execution_count AS ReadsMedios,
t.last_execution_time AS UltimaExecucao
FROM sys.dm_exec_trigger_stats t
WHERE t.database_id = DB_ID()
ORDER BY t.total_worker_time DESC;
GO
-- 3. Análise de waits relacionados a triggers
SELECT TOP (20)
wait_type,
waiting_tasks_count,
wait_time_ms / 1000 AS WaitSegundos,
signal_wait_time_ms / 1000 AS SignalSegundos,
wait_time_ms / NULLIF(waiting_tasks_count, 0) AS WaitMedioMs
FROM sys.dm_os_wait_stats
WHERE wait_type LIKE 'LCK%'   -- locks
OR wait_type LIKE 'LATCH%'   -- latches internos
ORDER BY wait_time_ms DESC;
GO
-- 4. Padrão assíncrono via Service Broker (advanced)
-- Em vez de INSERT direto no log, enviar mensagem
-- para fila processada por job separado
-- (código avançado, fora do escopo deste slide)
```

DECISÃO DE ARQUITETURA

Alternativas Modernas a Triggers

Cada alternativa cobre um caso de uso — escolha a mais simples que resolve o problema.

5 ALTERNATIVAS · 8 ASPECTOS
DO SIMPLES AO COMPLEXO

ASPECTO	SIMPLES	DERIVAÇÃO	AUDITORIA	ASSÍNCRONO	APP
Aspecto	CHECK Constraint	Computed Column	Temporal Tables	Change Data Capture	Application Layer
Caso de uso	Validação simples	Derivação de dados	Histórico de alterações	Auditoria assíncrona	Lógica complexa
Disponível desde	Sempre	Sempre	SQL 2016	SQL 2008	N/A
Performance	✓ Excelente	✓ Excelente	✓ Muito boa	✓ Boa (assíncrono)	⚠ Depende
Atomicidade	✓ Sim	✓ Sim	✓ Sim	⚠ Assíncrono	⚠ Não (cross-layer)
Visibilidade	✓ Explícita	✓ Explícita	✓ Automática	✓ Automática	✓ No código
Setup	Simples	Simples	Médio	Médio	N/A
Limitações	Só coluna individual, sem consulta externa	Sem side-effects	Schema fixo após habilitar	Requer SQL Agent	Pode ser bypassed por scripts ad-hoc
Quando preferir	Regras simples (idade ≥ 18)	Total = soma, data formatada	Auditoria de tabela inteira	Sincronização com DW, ETL	Lógica de workflow, validação UX

→ Regra geral: tente primeiro `CHECK` → `Computed` → `Temporal` → `CDC` → `Trigger`. Use trigger apenas como último recurso.

CHECKLIST PROFISSIONAL

✓ Checklist Profissional de Triggers

Imprima e cole na parede — antes de escrever trigger, pergunte-se.

12 ITENS · 3 EIXOS

ALTERNATIVAS → IMPLEMENTAÇÃO → OPERAÇÃO

✓ CHECKLIST PROFISSIONAL · 12 ITENS

v1.0 · AULA 04

- 1✓ Já tentei CHECK constraint? — Validação simples NÃO precisa de trigger
- 2✓ Já considerei Computed Column? — Derivação de dados NÃO precisa de trigger
- 3✓ Já considerei Temporal Table? — Auditoria NÃO precisa de trigger (SQL 2016+)
- 4✓ Já considerei FOREIGN KEY CASCADE? — Integridade simples NÃO precisa de trigger
- 5✓ A regra é cross-table e procedural? — Apenas aqui trigger se justifica
- 6✓ Set-based em TODO o corpo — Nunca cursor
- 7✓ SET NOCOUNT ON na primeira linha — Sempre, em toda trigger
- 8✓ IF UPDATE(col) + WHERE i.X <> d.X — Loga só mudanças reais
- 9✓ TRY/CATCH com log separado? — Erro da trigger não pode quebrar DML
- 10✓ Documentei via sp_addextendedproperty? — Metadados visíveis no catálogo
- 11✓ Monitorei sys.dm_exec_trigger_stats? — CPU/duração/reads por trigger
- 12✓ Mapeei via sys.sql_expression_dependencies? — Evitar quebras silenciosas

✦ SÍNTESE DA AULA 04

O que vimos hoje

- 4.1 · Implementação: tipos (DML/DDL/LOGON), anatomia, inserted/deleted, AFTER vs INSTEAD OF, três casos de uso (regras de negócio, auditoria, integridade), DDL e LOGON triggers, exemplo integrador.
- 4.2 · Boas Práticas: por que são problemáticas, custos (CPU/locks/memória), cascata, alternativas (CHECK/Computed/Temporal/CDC), otimização, monitoramento, checklist profissional.

✦ DESAFIO · AULA 05

hands-on

```
-- Desafio para Aula 05:
-- 1. Crie trigger trg_AuditarPedido em dbo.Pedidos
-- 2. Use a técnica de IF UPDATE(coluna)
-- 3. Log em dbo.Auditoria (genérica)
-- 4. Try/Catch com log de erro separado
-- 5. Compare com Temporal Table equivalente
```

→ **Aula 05:** Functions avançadas · XML/JSON nativos · In-Memory OLTP (Hekaton)